Fine-Tuning a Small Decoder for Clinical-Trial Criterion Eligibility, with a From-Scratch KV Cache, Tiled Attention and Speculative Decoding

Seerat E Zahra (23i-0640)
FAST-NUCES Islamabad, Generative AI – Assignment 2
Code: https://github.com/Seeratzahra500/GenAI-Assignment2
Adapters (Hugging Face): https://huggingface.co/Seerat500/trialgpt-qwen2.5-1.5b-lora
https://huggingface.co/Seerat500/trialgpt-qwen2.5-0.5b-draft-lora

***Abstract*—We fine-tune Qwen2.5-1.5B-Instruct with LoRA on the expert annotations of TrialGPT-Criterion-Annotations to predict, for a patient note and one trial criterion, an eligibility label and the supporting sentence IDs. Using a patient-grouped 37/8/8 split we compare the base and fine-tuned models with criterion-level metrics, and analyse errors. We also implement, and benchmark against references, a KV-cache decoder, a tiled online-softmax attention kernel, and greedy speculative decoding with a LoRA-tuned Qwen2.5-0.5B draft, and expose everything in a Gradio app. Fine-tuning raises test accuracy from 0.107 to 0.748 and evidence micro-F1 from 0.182 to 0.536. Rare classes remain weak, and speculative decoding was slower than cached greedy decoding in our setting; we report both honestly. The prototype is educational and not a clinical decision tool.**

***Index Terms*—LoRA, clinical trial matching, KV cache, online softmax, speculative decoding**

I. Introduction

Matching patients to clinical trials requires judging each inclusion and exclusion criterion against a free-text note. TrialGPT [1] showed that LLMs can do this and released expert criterion-level annotations. We study whether a small open-weight decoder, adapted with LoRA [2] on a single 16 GB GPU, can learn the task, and we implement the decoding machinery (KV cache, tiled attention [3], [4], speculative decoding [5], [6]) ourselves so that every component can be verified against a reference.

II. Related Work

Clinical trial matching. TrialGPT [1] decomposes matching into criterion-level eligibility labels with evidence sentences and uses GPT-4 prompting; its expert annotations are the benchmark used here, while its GPT-4 outputs are never used as input or for evaluation. Prompting a large proprietary model is accurate but not locally executable, so we instead adapt a small open-weight decoder. **Parameter-efficient adaptation.** LoRA [2] freezes the base weights and trains low-rank updates, which fits a 1.5B model on one 16 GB GPU; QLoRA [7] additionally quantises the base weights, which we did not need at this scale. **Efficient attention.** Standard attention materialises a $T \times T$ score matrix. Online softmax [4] and its blocked use in memory-efficient attention [8] and FlashAttention [3] compute exact attention with $O(T)$ extra memory; FlashAttention fuses this into one GPU kernel. Our PyTorch version reproduces the algorithm for verification, not for speed. **Inference.** Caching keys and values removes the quadratic recomputation during generation [9]; grouped-query attention [10] shrinks the cache (Qwen2.5-1.5B has 2 KV heads for 12 query heads), and rotary position embeddings [11] make explicit position handling necessary when a cache is reused. Speculative decoding [5], [6] lets a small draft model propose tokens that the target verifies in one pass, giving the target's exact greedy output; gains depend on the draft/target cost ratio and acceptance rate.

III. Data and Split

We use only the expert columns (`expert_eligibility`, `expert_sentences`); the GPT-4 columns and the original `training` flag are never read. The dataset has 1015 criterion annotations from 53 patients. Patients (not rows) are split to avoid leakage: sorted unique patient IDs are shuffled with `numpy.random.default_rng(42)` and divided 37/8/8, giving 603/206/206 annotations; assertions confirm that no patient appears in two splits and the manifest is committed. Table I shows the label distribution, which is strongly imbalanced: some classes have only 1–3 examples in validation and test, so per-class numbers are noisy. Prompts plus answers have at most 562 tokens (mean 267), so no truncation occurs at the 1024-token limit.

IV. Model Selection and Resources

We required an open-weight, locally executable decoder that fits LoRA training on a free Kaggle Tesla T4 (16 GB) and has a smaller sibling with the same tokenizer for speculative decoding. Table II compares candidates. Qwen2.5-1.5B [12] was chosen because it is Apache-2.0, small enough for fp16 LoRA training with gradient checkpointing, supports a 32k context (our longest example is 562 tokens), and shares its 151,936-token vocabulary with Qwen2.5-0.5B, which makes the 0.5B model a valid draft (the code asserts identical tokenizers). Its KV cache costs $2 \cdot 28 \cdot 2 \cdot 128 \cdot 2 = 28{,}672$

TABLE I
LABEL COUNTS PER SPLIT (INCLUSION / EXCLUSION CRITERIA).

Type	Label	Train	Val	Test
inclusion	included	88	35	27
inclusion	not included	13	3	7
inclusion	not enough information	109	27	38
inclusion	not applicable	5	1	3
exclusion	excluded	8	2	0
exclusion	not excluded	292	101	91
exclusion	not enough information	65	26	28
exclusion	not applicable	23	11	12
All annotations		603	206	206

bytes per token (28 layers, 2 KV heads, head dimension 128, fp16), i.e. about 4 MiB for 144 tokens, which matches the 3.94 MiB we measured. **Resources.** All experiments ran on a Kaggle Tesla T4 with 16 GB; fp16 weights occupy about 2.9 GiB for the target and about 3.8 GiB with the draft loaded. LoRA training took about 18 min (v1) and 25 min (v2). A batch of 4 ran out of memory, so training uses batch 1 with 16 accumulation steps and gradient checkpointing. No paid service was used.

A. Alternatives, difficulties and decisions

(i) We trained a second variant that oversamples rare classes threefold (v2); it helps evidence precision but not accuracy, and v1 stays final. (ii) fp16 can flip argmax on near-tied logits, so cached and uncached outputs can diverge although both are correct; we confirmed this with an fp32 re-run. (iii) A first cache-based speculative implementation raised an error in `DynamicCache.crop` when given a positive length; cropping is now done through a small helper that crops by the number of rejected positions. (iv) A dependency conflict between `torchao` and PEFT required removing `torchao`.

V. METHOD

A. Task format

Fig. 1 summarises the system.

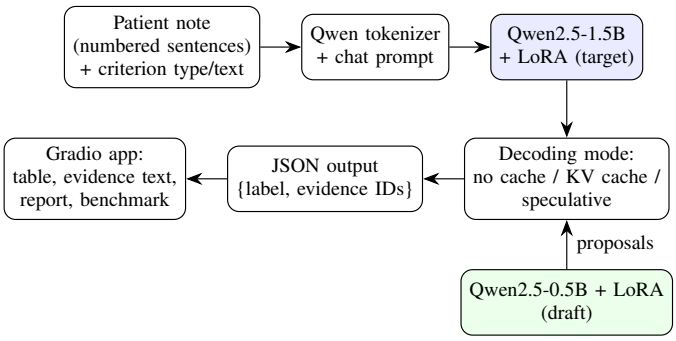


Fig. 1. System overview: prompt construction, the LoRA-tuned target, the three decoding paths (the draft is used only in speculative mode) and the application.

A chat prompt contains the numbered note, the criterion type and text, the allowed labels for that type, and asks for JSON only: `{"label": ..., "evidence": [ids]}`. Inclusion labels are *included, not included, not enough information, not applicable*; exclusion labels are *excluded, not excluded, not enough information, not applicable*. Outputs are parsed strictly; unparsable or out-of-vocabulary outputs are counted as the label "invalid" (wrong), never silently repaired.

B. LoRA fine-tuning

The target is Qwen2.5-1.5B-Instruct [12]. LoRA adapters (rank 16, α=32, dropout 0.05) are placed on all linear projections. Loss is computed on the answer tokens only. Settings: lr 2×10^{-4} with cosine decay and 6 warm-up steps, 3 epochs, per-device batch 1 with 16 accumulation steps, fp16 autocast with fp32 adapter weights, gradient checkpointing, seed 42. The checkpoint with the lowest validation loss is kept (step 80). A second variant (v2) oversamples rare classes; it was trained for comparison and is reported below. A 0.5B draft adapter was trained with the same recipe for speculative decoding. Fig. 2 shows the training curves.

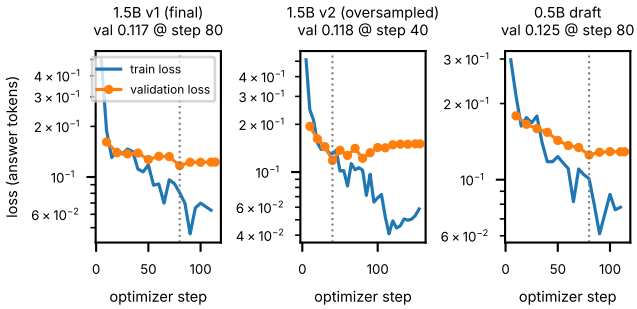


Fig. 2. Training and validation loss (log scale); dotted line marks the selected checkpoint.

C. Metrics

Overall and per criterion type: accuracy, macro-F1 (over classes present in the gold labels of that split), per-class precision/recall/F1 and confusion matrices. Evidence is scored on sentence-ID sets with micro P/R/F1 over all records and a per-record macro average in which an empty gold set matched by an empty prediction counts as correct. Because many records have empty gold evidence, we also report micro metrics restricted to non-empty gold.

VI. RESULTS

A. Experimental setup

All models are evaluated on identical validation and test records with greedy decoding (`do_sample=False`), at most 64 new tokens, batch size 8, fp16; a full test pass takes about 27 s on the T4. The same prompt is used before and after fine-tuning, so the base model's low score also reflects format-following failures (parse failures count as wrong). Records with empty gold evidence are handled as described under Metrics.

TABLE II
CANDIDATE DECODERS (WEIGHTS IN FP16 ≈ 2 BYTES/PARAMETER; LLAMA FIGURES FROM ITS MODEL CARD).

Model	Params	Layers	Q/KV heads	Context	fp16 weights	Licence	Verdict
Qwen2.5-1.5B-Instruct	1.54B	28	12/2	32768	3.1 GB	Apache-2.0	**chosen (target)**
Qwen2.5-0.5B-Instruct	0.49B	24	14/2	32768	1.0 GB	Apache-2.0	chosen (draft)
Llama-3.2-1B-Instruct	1.24B	16	32/8	131072	2.5 GB	Llama community	rejected: gated, different tokenizer family
Phi-3-mini-4k-instruct	3.82B	32	32/32	4096	7.6 GB	MIT	rejected: LoRA training too tight on 16 GB, no small same-tokenizer draft

B. Before vs. after fine-tuning

Table III (validation) and Table IV (test, evaluated once after model selection) compare the base model with fine-tuned variants. The final model is v1 (plain LoRA), chosen on validation accuracy and validation loss before any test evaluation. The oversampled variant v2 has slightly higher validation macro-F1 and evidence F1 but lower accuracy; on test it again trades accuracy (0.714 vs 0.748) and exclusion macro-F1 for higher evidence precision. Given supports of 1–3 for rare classes these differences are within noise, and we did not switch models after seeing test results. Evidence results are in Table V; the high per-record macro F1 is inflated by records whose gold evidence is empty.

TABLE III
VALIDATION RESULTS.

Model	Acc.	Macro-F1	Incl. acc.	Incl. F1	Excl. acc.	Excl. F1	Evid. F1
Base (no fine-tuning)	0.107	0.099	0.303	0.147	0.014	0.007	0.128
v1 LoRA (final)	0.752	0.445	0.742	0.383	0.757	0.360	0.525
v2 LoRA (oversampled)	0.723	0.467	0.712	0.379	0.729	0.377	0.582

TABLE IV
TEST RESULTS (SINGLE EVALUATION).

Model	Acc.	Macro-F1	Incl. acc.	Incl. F1	Excl. acc.	Excl. F1	Evid. F1
Base (no fine-tuning)	0.107	0.141	0.293	0.185	0.000	0.000	0.182
v1 LoRA (final)	0.748	0.550	0.787	0.426	0.725	0.458	0.536
v2 LoRA (oversampled)	0.714	0.511	0.747	0.482	0.695	0.275	0.584

TABLE V
EVIDENCE-SENTENCE METRICS, TEST SPLIT.

	All records (micro)			Macro	Non-empty gold only (micro)		
Model	P	R	F1	F1	P	R	F1
Base (no fine-tuning)	0.105	0.702	0.182	0.194	0.400	0.702	0.510
v1 LoRA (final)	0.551	0.521	0.536	0.809	0.766	0.521	0.620
v2 LoRA (oversampled)	0.750	0.479	0.584	0.841	0.818	0.479	0.604

C. Per-class behaviour

Table VI and Fig. 3 show per-class scores and confusion matrices on the test split for the final model. Majority classes (*included*, *not excluded*, inclusion *not enough information*) are learned. The clearest failure is exclusion *not enough information*: recall is 0.04 (28 gold cases, almost all predicted *not excluded*), so the model treats missing information as absence of the exclusion condition. *Not included* (7) and inclusion *not applicable* (3) are never predicted correctly. The test split contains no *excluded* case, so that class has no support there (validation had 2, also missed). Classes with a handful of examples are unreliable, since one example moves their F1 in large steps.

TABLE VI
PER-CLASS RESULTS, FINAL MODEL, TEST.

Type	Label	P	R	F1	Support
inclusion	included	0.96	0.81	0.88	27
	not included	0.00	0.00	0.00	7
	not enough information	0.71	0.97	0.82	38
	not applicable	0.00	0.00	0.00	3
exclusion	excluded	n/a	n/a	n/a	0
	not excluded	0.72	0.99	0.83	91
	not enough information	1.00	0.04	0.07	28
	not applicable	0.80	0.33	0.47	12

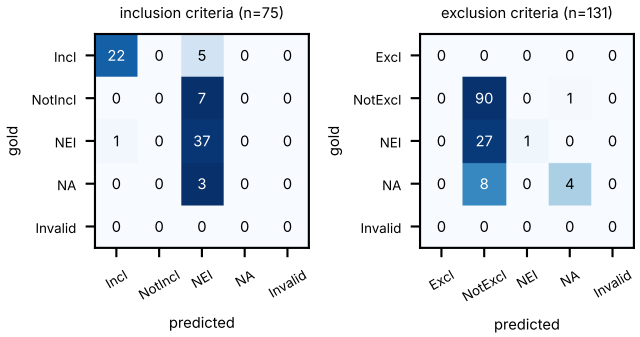


Fig. 3. Test confusion matrices (counts; colour is row-normalised).

D. Error analysis

Table VII lists twelve test errors chosen automatically from distinct (gold, predicted) groups, at most two per group and patient. The recurring categories are: (i) a failed inclusion criterion not recognised, so *not included* becomes *not enough information* (rows 1–2); (ii) missing information treated as absence, so *not enough information* becomes *not excluded*; (iii) non-applicability not recognised, e.g. breast-feeding, birth-control or prior-tonsillectomy criteria answered as *not excluded* or *not enough information* (rows 3–6); (iv) correct label with wrong or missing cited sentences (rows 11–12); (v) over-caution, where an *included* criterion is answered *not enough information* (row 10), and the opposite over-confidence, where *not enough information* is answered *included* (row 9). In several rows the model cites no sentence at all although the gold evidence is non-empty, which matches the low evidence recall (0.52).

TABLE VII
TEST ERRORS OF THE FINAL MODEL (FULL TEXT IN REPORT/GENERATED/ERRORS.JSON).

#	Type	Gold → predicted	Criterion	Gold ev.	Pred. ev.	Category
1	incl.	not included → not enough information	Patients undergoing elective carotid endarterectomy	–	0	contradicting evidence missed (contradictory case)
2	incl.	not included → not enough information	The use of vasopressors to maintain a systolic blood pres...	3	–	contradicting evidence missed (contradictory case)
3	excl.	not applicable → not excluded	Previous tonsillectomy, including partial tonsillectomy	–	–	non-applicability not recognised
4	excl.	not applicable → not excluded	breast-feeding	–	–	non-applicability not recognised
5	incl.	not applicable → not enough information	If female, using birth control	3	–	non-applicability not recognised
6	incl.	not applicable → not enough information	female patients have adequate birth-control method	–	–	non-applicability not recognised
7	excl.	not enough information → not excluded	Patients with an estimated pre-operative glomerular filtr...	–	–	missing information treated as absence or eligibility
8	excl.	not enough information → not excluded	Patients were moribund.	0,1,2,3,4	–	missing information treated as absence or eligibility
9	incl.	not enough information → included	A sore and scratchy throat and/or pain on swallowing (ody...	0	0,2	missing information treated as absence or eligibility
10	incl.	included → not enough information	No medical history of central vena catheterization.	0	–	supporting evidence not found (over-cautious)
11	incl.	not enough information → not enough information	referring doctor suspects peritonsillar abscess	0	–	correct label, cited sentences miss the gold evidence
12	incl.	not enough information → not enough information	Normal lung function, defined as (Knudson 1976/1984 predi...	0	–	correct label, cited sentences miss the gold evidence

VII. KV-Cache Decoder

`generate_cached` runs an explicit prefill and then one token per step, passing explicit `position_ids`, a growing attention mask and the carried cache; `generate_nocache` recomputes the full sequence each step and is the reference. Table VIII compares them (greedy, batch 1, fixed number of new tokens, fp16, one warm-up run and the median of three timed runs with CUDA synchronisation). The speed-up grows with input length (1.1× at 128 tokens, 4.5× at 512 with 64 new tokens) because the reference cost grows with sequence length while a cached step does not; peak memory differs little at these lengths because weights dominate, and the KV cache itself is small (GQA with 2 KV heads). In one configuration (256 in / 64 new) the token streams diverge at step 20; an fp32 re-run was identical, so the divergence is an fp16 near-tie in the argmax, not a cache bug.

VIII. Tiled Attention

`tiled_attention` processes queries and keys in blocks keeping a running row maximum m, normaliser ℓ and accumulator a: for each key block with scores S, $m' = \max(m, \max S)$, $\ell \leftarrow \ell e^{m-m'} + \sum e^{S-m'}$, $a \leftarrow a e^{m-m'} + e^{S-m'} V$, and the output is a/ℓ [4]. It never materialises the full $T_q \times T_k$ matrix, skips fully masked causal blocks, supports $T_q < T_k$ via an offset, and guards $-\infty - (-\infty)$. Table IX cross-checks it against a stable reference softmax and PyTorch SDPA (28 cases, worst tiled-vs-reference error 6.2×10^{-6}). Table X and Fig. 4 give timing and peak extra memory. Our tiled version is written in PyTorch loops, so it is not expected to beat the fused SDPA kernel in time; its value is the memory behaviour and the verified algorithm.

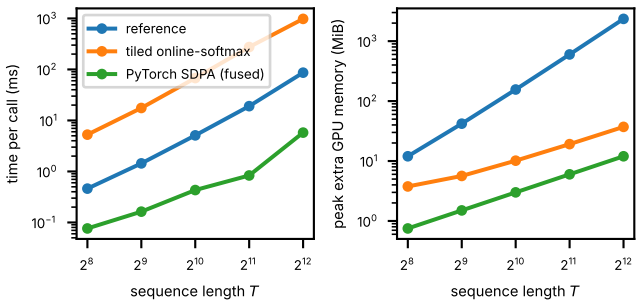


Fig. 4. Attention benchmark.

IX. Speculative Decoding

A 0.5B draft (same tokenizer, asserted in code) proposes k tokens greedily; the 1.5B target verifies [last] + drafts in one pass, the longest agreeing prefix is accepted, and the target's own next token is appended as correction or bonus token. Both caches are cropped to hold exactly the accepted sequence; the invariant at the start of a round is that both caches hold KV for all tokens but the last. Output is identical to the target's greedy output (8/8 prompts equal). Table XI shows speculative decoding was *slower* than cached greedy decoding (0.65–0.76×): with a 1.5B target the draft is only 3× smaller, acceptance is 0.5–0.7, and each round pays draft and verification overhead plus cache cropping in Python, whereas

TABLE VIII
KV CACHE VS. NO-CACHE REFERENCE.

T_{in}	n	Tokens equal	Prefill ms (no cache / cache)	Decode tok/s (no cache)	Decode tok/s (cache)	Total tok/s (no cache / cache)	Speed-up	Peak MiB (no cache / cache)	KV MiB
128	16	yes	44 / 45	22.7	25.5	22.7 / 25.3	1.1×	3014 / 3016	3.9
128	64	yes	44 / 45	22.8	25.1	22.8 / 25.1	1.1×	3102 / 3083	5.2
256	16	yes	58 / 47	13.9	25.0	14.0 / 24.7	1.8×	3125 / 3096	7.4
256	64	no (step 20)	60 / 49	12.6	24.6	12.7 / 24.5	1.9×	3139 / 3124	8.8
512	16	yes	130 / 117	6.3	25.7	6.4 / 22.9	3.6×	3200 / 3182	14.4
512	64	yes	147 / 137	5.5	25.7	5.5 / 24.7	4.5×	3214 / 3210	15.8

TABLE IX
TILED ATTENTION CORRECTNESS (FP32).

Shape (B, H, T_q, T_k, D)	Max abs. err.	Mean abs. err.	Max err. vs SDPA
(1, 2, 8, 8, 16)	2.4×10^{-7}	1.5×10^{-8}	2.4×10^{-7}
(2, 4, 37, 37, 32)	6.2×10^{-6}	5.5×10^{-8}	6.2×10^{-6}
(1, 4, 64, 64, 64)	7.2×10^{-7}	2.5×10^{-8}	7.2×10^{-7}
(2, 2, 100, 100, 64)	9.5×10^{-7}	2.7×10^{-8}	9.5×10^{-7}
(1, 2, 1, 50, 32)	4.8×10^{-7}	5.1×10^{-8}	7.2×10^{-7}
(1, 2, 5, 50, 32)	4.8×10^{-7}	3.4×10^{-8}	4.8×10^{-7}
(1, 8, 257, 257, 64)	1.2×10^{-6}	2.8×10^{-8}	9.5×10^{-7}

TABLE X
ATTENTION TIMING / MEMORY (BATCH 1, 12 HEADS, D=128).

T	Time (ms)			Peak extra memory (MiB)		
	ref.	tiled	SDPA	ref.	tiled	SDPA
256	0.46	5.27	0.08	12.0	3.8	0.8
512	1.44	17.55	0.16	42.0	5.6	1.5
1024	5.10	67.75	0.43	156.0	10.1	3.0
2048	19.06	274.91	0.83	600.0	19.1	6.0
4096	86.48	980.06	5.77	2352.0	37.1	12.0

a cached 1.5B step is already cheap. Larger target/draft ratios, higher acceptance or batched verification would be needed for a gain.

TABLE XI
SPECULATIVE VS. GREEDY CACHED DECODING (48 NEW TOKENS, 8 PROMPTS).

Mode	Tokens equal	tok/s	Speedup	Accept. rate	Acc./round	Target passes	Peak MiB
Greedy + KV cache	–	25.5	1.00×	–	–	–	4030
Speculative $k = 2$	8/8	19.5	0.76×	0.69	1.34	169	4088
Speculative $k = 4$	8/8	17.9	0.70×	0.58	2.19	126	4088
Speculative $k = 6$	8/8	16.5	0.65×	0.50	2.84	106	4088

X. APPLICATION

The Gradio application has three tabs. (1) *Screening*: the user pastes a patient description (plain text or dataset-style numbered sentences) and enters inclusion and exclusion criteria, one per line; a dropdown selects the decoding mode (KV cache, no-cache reference, speculative with adjustable k). The result shows, per criterion, the predicted category, cited sentence IDs, the exact evidence text, the raw model output and the time taken, followed by a preliminary outcome computed by an explicit rule (not a ground-truth verdict), and a downloadable Markdown report with provenance (models, adapters, decoding mode, timestamp, sentence handling). (2) *Inference benchmark*: runs the first criterion live through no-cache, KV-cache and speculative decoding and reports time, tokens/s, speed-up, peak GPU memory, token equality with the reference, acceptance rate and target passes. (3) *Held-out test results*: loads the stored test metrics. Invalid inputs (empty or too short notes, over-long input) raise a clear message and the app stays usable. A prominent notice states that this is an educational prototype, not a clinical enrolment decision (Figs. 5 and 6).

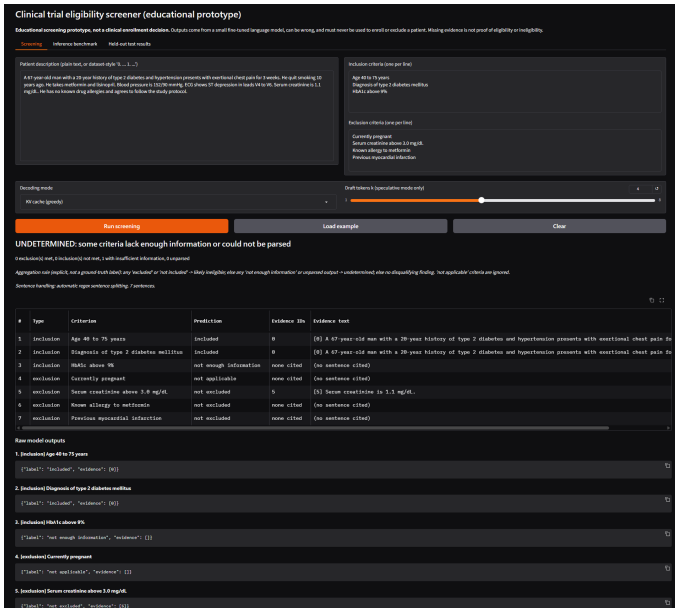


Fig. 5. Screening tab: inputs, decoding mode, preliminary outcome and aggregation rule, and the per-criterion table with predictions, cited sentence IDs and evidence text.

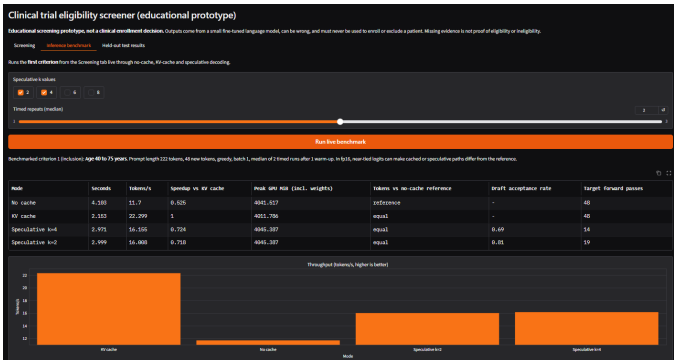


Fig. 6. Inference benchmark tab: the first criterion decoded live without cache, with the KV cache and with speculative decoding ($k = 2, 4$); tokens match the no-cache reference in every mode.

Startup. The app is launched locally with `python app/app.py` and opened at `http://127.0.0.1:7860`; it downloads the base models and the published adapters and uses the GPU if present, otherwise the CPU. No notebook is needed. Exact steps are in the repository README.

XI. Limitations

Only 53 patients; validation/test supports of 1–3 for rare classes make per-class scores noisy and differences between v1 and v2 within noise. Model selection used the validation split and test was evaluated once for the final model. Greedy fp16 decoding can differ across hardware on near-ties. The public app link exists only while the notebook session runs. No clinical validation was performed.

XII. Conclusion

LoRA fine-tuning makes a 1.5B decoder usable for criterion-level eligibility labelling (test accuracy 0.107 $\rightarrow$ 0.748, macro-F1 0.141 $\rightarrow$ 0.550), but rare classes, exclusion criteria with missing information, and evidence selection remain hard. The KV cache gives up to 4.5$\times$ speed-up at 512 tokens, tiled attention matches the reference to float precision, and speculative decoding preserved outputs but did not speed things up here.

Appendix A
AI-Use Statement

I used Anthropic's Claude as a coding and writing assistant. It helped plan the pipeline and model choice, wrote first drafts of the training, evaluation, KV-cache, tiled-attention, speculative-decoding and Gradio code and of the figure/table scripts, and drafted the first version of this text. I ran every training, evaluation and benchmark job myself on Kaggle (Tesla T4); all numbers in this report come from the JSON files in `outputs/` produced by those runs, not from AI-written text. I selected the final model on validation metrics before looking at the test set. When AI-written code failed (missing-file and path errors, a `torchao`/`peft` conflict, CPU/GPU device handling in the app) I re-ran it and debugged it with the assistant until it worked. I tested the application, read the error examples against the stored predictions, and recorded the demo. I can explain and modify each component in the live defense.

References

[1] Q. Jin *et al.*, "Matching patients to clinical trials with large language models," *Nature Communications*, vol. 15, 2024.
[2] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," in *ICLR*, 2022.
[3] T. Dao *et al.*, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *NeurIPS*, 2022.
[4] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," arXiv:1805.02867, 2018.
[5] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *ICML*, 2023.
[6] C. Chen *et al.*, "Accelerating large language model decoding with speculative sampling," arXiv:2302.01318, 2023.
[7] T. Dettmers *et al.*, "QLoRA: Efficient finetuning of quantized LLMs," in *NeurIPS*, 2023.
[8] M. N. Rabe and C. Staats, "Self-attention does not need $O(n^2)$ memory," arXiv:2112.05682, 2021.
[9] R. Pope *et al.*, "Efficiently scaling transformer inference," in *MLSys*, 2023.
[10] J. Ainslie *et al.*, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *EMNLP*, 2023.
[11] J. Su *et al.*, "RoFormer: Enhanced transformer with rotary position embedding," arXiv:2104.09864, 2021.
[12] A. Yang *et al.*, "Qwen2.5 technical report," arXiv:2412.15115, 2024.